

Задание

Разработать программу для анализа access log веб-сервера

Программа получает журнал HTTP-запросов, строит статистику по клиентам, статусам и ресурсам, а также сравнивает использование ordered и unordered containers для разных видов поиска

Путь к исходному файлу передается через аргумент командной строки

Пример запуска:

```
./log_analyzer access.log
```

Каждая строка входного файла содержит один запрос:

```
ip method path status bytes
```

Например:

```
203.0.113.15 GET /catalog 200 13420
198.51.100.4 GET /login 401 821
203.0.113.15 POST /cart 200 532
192.0.2.77 GET /catalog 200 11840
203.0.113.15 GET /profile 500 256
```

Поля имеют следующий смысл:

Поле	Значение
ip	IP-адрес клиента
method	HTTP method
path	путь ресурса
status	HTTP status code
bytes	размер ответа в байтах

Формат строк можно считать корректным. Файл может содержать несколько тысяч строк

Использовать следующие HTTP методы:

```
GET
POST
PUT
DELETE
PATCH
```

Программа должна построить статистику:

- общее количество запросов
- количество уникальных IP-адресов
- количество запросов по каждому HTTP status
- количество запросов к каждому path
- суммарный объем ответов по каждому IP-адресу
- количество запросов по каждому IP-адресу
- топ-10 самых частых ресурсов

- топ-10 клиентов по количеству запросов
- количество запросов по составному ключу `method + path`

Для хранения и анализа данных использовать associative containers:

- `std::set` для уникальных IP-адресов в отсортированном порядке
- `std::unordered_set` для быстрой проверки, встречался ли IP-адрес
- `std::map` для статистики по HTTP status и отсортированного вывода
- `std::unordered_map` для счетчиков по path, IP-адресам и составному ключу

Для составного ключа `method + path` создать отдельный пользовательский тип. Для этого типа нужно реализовать сравнение для ordered container и hash для unordered container

В программе должно быть показано, что ordered containers хранят ключи в определенном порядке, а unordered containers используют hash и не задают порядок обхода элементов

При обработке map-like containers использовать structured bindings для пары key/value

Программа должна сохранить итоговый отчет в файл `report.txt`

Программа должна вывести отчет:

- общее количество запросов
- количество уникальных IP-адресов
- статистику по статусам в порядке возрастания status code
- первые 10 IP-адресов из отсортированного множества
- топ-10 ресурсов по количеству запросов
- топ-10 клиентов по количеству запросов
- статистику по нескольким значениям `method + path`
- суммарный объем ответов для 5 клиентов с наибольшим числом запросов

Полноценный интерактивный CLI реализовывать не требуется. Достаточно прочитать путь к входному файлу из аргументов командной строки, вывести отчет и сохранить его в `report.txt`

Требования

- Названия переменных, функций, типов и других сущностей программы должны быть осмысленными и записываться на английском языке без транслитерации
- Программа должна читать путь к входному файлу из аргументов командной строки
- Программа должна обрабатывать запуск без аргумента командной строки
- В программе должны использоваться `std::set`, `std::map`, `std::unordered_set` и `std::unordered_map`
- Для составного ключа `method + path` должен быть реализован отдельный пользовательский тип
- Для пользовательского ключа должен быть реализован comparator или операция сравнения для ordered container
- Для пользовательского ключа должен быть реализован hash для unordered container
- В программе должны использоваться `std::pair` и structured bindings при обработке пар ключ-значение
- Программа должна показывать отличие ordered и unordered containers при обходе элементов
- Программа должна сохранять итоговый отчет в файл `report.txt`

- Программа должна быть грамотно разбита на модули и содержать как минимум один `.hpp` файл и два `.cpp` файла. Допускается использовать больше файлов
- Для заголовочных файлов использовать `#pragma once`
- Программа должна компилироваться и выполняться без ошибок
- Студент должен понимать написанный код, уметь обосновать свое решение и отвечать на вопросы во время защиты лабораторной работы

Минимальный набор операций:

- чтение записи `access log` из входного потока
- добавление записи в набор статистик
- подсчет запросов по `path`
- подсчет запросов по составному ключу `method + path`
- формирование топ-10 элементов по счетчику
- запись итогового отчета в выходной поток